

Práctica 1

Memoria compartida (C++, OpenMP)

Instrucciones para la entrega:

Para la evaluación de esta práctica se debe entregar el código desarrollado y una memoria en formato PDF contestando a las preguntas del enunciado y mostrando los bloques de código relevantes, acompañados de una explicación de dicho código que describa como resuelve el problema planteado.

Para la entrega de dicha práctica se deberán subir los archivos correspondientes a las diferentes entregas creadas para ello: 1 entrega para la memoria, otra entrega para el proyecto de código, comprimido en formato ZIP.

Contexto:

OpenMP es una interfaz de programación de aplicaciones (API) para la programación multiproceso de memoria compartida en múltiples plataformas. Permite añadir concurrencia a los programas escritos en C, C++ y Fortran sobre la base del modelo de ejecución fork-join. Está disponible en muchas arquitecturas, incluidas las plataformas de Unix y de Microsoft Windows. Se compone de un conjunto de directivas de compilador, rutinas de biblioteca, y variables de entorno que influyen el comportamiento en tiempo de ejecución. - [Wikipedia](#)

En este caso, la práctica se realizará a partir del proyecto CMake proporcionado en la actividad, y se utilizará C++ como lenguaje de programación para el desarrollo.

OpenMP

En esta práctica se van a explorar algunas de las instrucciones básicas de OpenMP, que permitirán ajustar los diferentes parámetros de configuración de OpenMP para la ejecución paralela del código.

```
#pragma omp
```

```
parallel for private(variable_name)
```

```
parallel for shared(variable_name)
```

```
reduction(+:variable_name)
```

```
schedule(static | dynamic | guided, job_size_iterations)
```

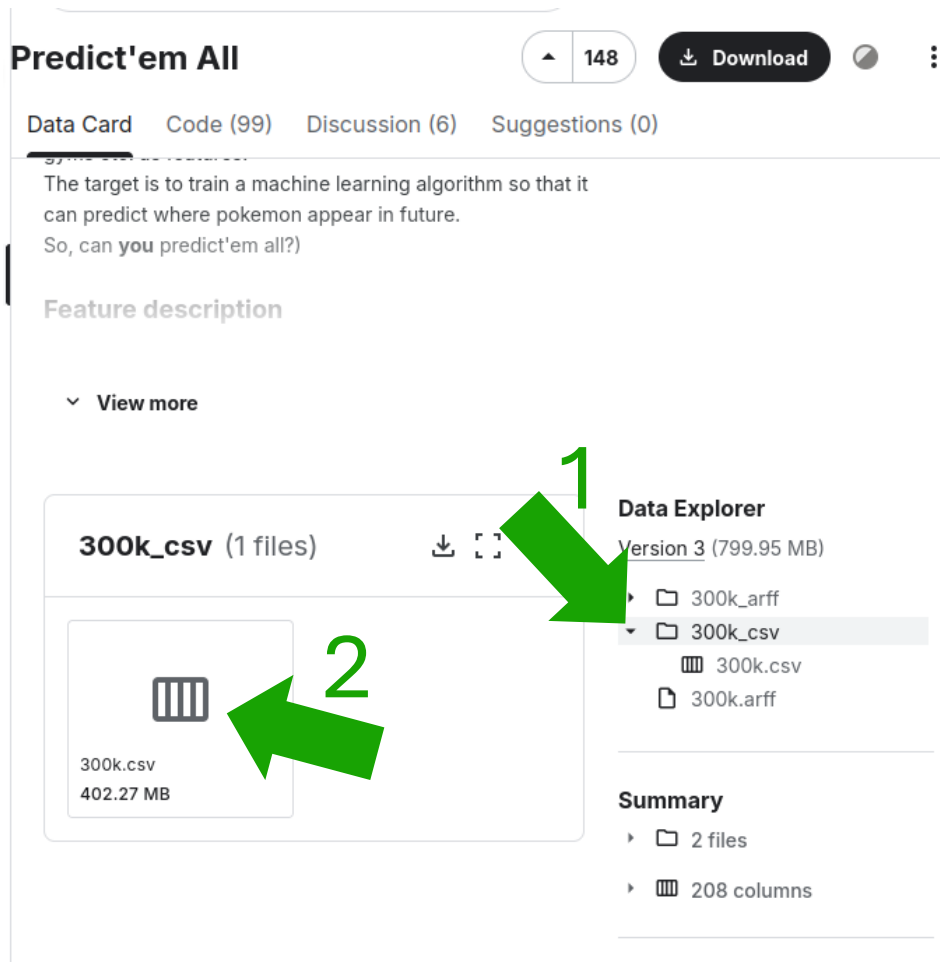
```
int omp_get_num_procs()
```

```
int omp_get_thread_num()
```

```
omp_set_num_threads(int)
```

Datos

La versión CSV de los datos utilizados para esta práctica se pueden descargar en el siguiente enlace:
<https://www.kaggle.com/datasets/semioniy/predictemall>



The screenshot shows the Kaggle dataset page for 'Predict'em All'. The page has a header with the dataset name, a 'Download' button, and a '148' badge. Below the header, there are tabs for 'Data Card', 'Code (99)', 'Discussion (6)', and 'Suggestions (0)'. The 'Data Card' tab is selected, showing a description of the dataset: 'The target is to train a machine learning algorithm so that it can predict where pokemon appear in future. So, can you predict'em all?'. Below the description, there is a 'Feature description' section with a 'View more' link. The main content area shows the dataset files: '300k_csv (1 files)'. A green arrow labeled '1' points to the 'Data Explorer' section, which shows the dataset structure: 'Version 3 (799.95 MB)' containing '300k_arff', '300k_csv', '300k.csv', and '300k.arff'. Another green arrow labeled '2' points to the '300k.csv' file, which is shown with a file icon and the text '300k.csv 402.27 MB'. A 'Summary' section at the bottom shows '2 files' and '208 columns'.

En este repositorio se recogen datos de aproximadamente 296.000 avistamientos de Pokémon, recogidos de jugadores del juego de móvil Pokémon Go. Estos datos recogen, entre otras variables, el identificador de dicho Pokémon, el lugar de aparición, datos climáticos y de ambiente, datos de correlación con otros Pokémon y distancia a otros gimnasios.

Bibliografía y enlaces de interés:

- <https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf> (Guía de referencia oficial)
- <https://es.wikipedia.org/wiki/OpenMP#> (Historia y comandos útiles de configuración)
- <https://610yilingliu.github.io/tags/#OpenMP>

Cuestiones:



1. Calcular en el histórico de registros proporcionado el número de apariciones de Pikachus utilizando un bucle mono-hilo y una reducción paralela para comparar sus tiempos en milisegundos (ms). Compara los resultados a lo largo de 10 ejecuciones de cada tipo. ¿Qué enfoque es más eficiente en este caso? Razona tu respuesta. **(2 puntos)**
2. Obtener todas las apariciones de cada uno de los Pokémon, el más frecuente y el más raro basándose en su ID mediante un bucle para una variable de tipo `std::vector<unsigned int>` utilizando una implementación enfoque mono-hilo y otra reducción paralela. Compara los resultados a lo largo de 10 ejecuciones de cada tipo. ¿Qué enfoque es más eficiente en este caso? Razona tu respuesta. **(2 puntos)**
3. Calcular cuál será el Pokémon más próximo en aparecer en las coordenadas de latitud y longitud (20.525750, -97.46000) en un radio de 30 km basándote en el número de las 1000 apariciones más recientes y las más frecuentes de todo el histórico.
 - Utiliza solo la mitad de los cores disponibles en tu dispositivo.
 - Almacenar las posiciones bidimensionales utilizando glm. Para calcular la distancia entre dos puntos hay que utilizar la función `compute::calculateDistance(glm::dvec2 coord1, glm::dvec2 coord2)` que se proporciona en `/include/libDomain/Compute.h`. **(4 puntos)**
- 4.a Utiliza tiles para distribuir el cómputo del ejercicio 4 en bloques de $\frac{1}{4}$ de los cores disponibles en el dispositivo. Por ejemplo, si el PC tiene 8 cores, haremos 4 bloques de 2 cores/hilos para cada tile. **(1 punto)**
- 4.b. Desenrolla el bucle por completo y compara los tiempos de ejecución en milisegundos con el apartado anterior. **(1 punto)**